



RAG

RAG = Retrieval-Augmented Generation

RAG is an AI architecture where an LLM does not answer only from its pre-trained knowledge.

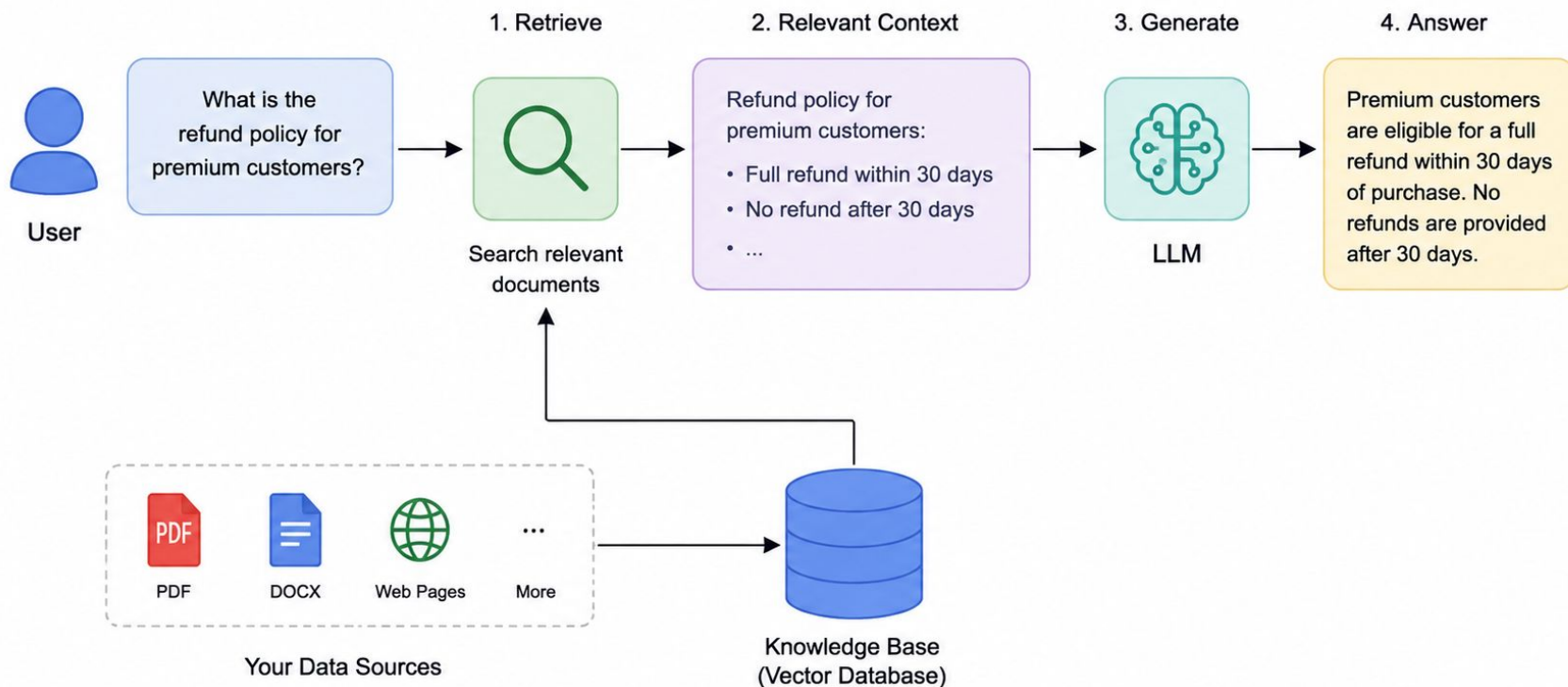
Instead, it first retrieves relevant information from external data sources like documents, PDFs, websites, databases, S3 files, knowledge bases, or vector databases, and then uses that information to generate a more accurate answer.

Simple flow:

User Question

- Search relevant documents
- Retrieve matching context
- Send context + question to LLM
- Generate grounded answer

RAG Pipeline



Core Components of a RAG Pipeline

A RAG pipeline has two major parts:

- **Data Preparation / Indexing Pipeline** - This prepares your business data so it can be searched later.
- **Retrieval + Generation Pipeline** : This runs when the user asks a question.

1. Data Sources

These are the places where your business knowledge is stored. **Examples:** PDFs, Word files, webpages, S3 files, databases, Confluence pages, product docs, support tickets, policies, reports

In real projects, the quality of RAG depends heavily on the quality of these data sources.

2. Data Ingestion

Data ingestion means collecting data from different sources and bringing it into your RAG pipeline.

Example:

- Files from Amazon S3
- Website content from URLs
- Customer policy documents from internal systems
- Product data from databases

From a Data Engineering POV, this step is very important because you need to handle: File formats, schema changes, incremental ingestion, duplicates, metadata, and data quality

Core Components of a RAG Pipeline

3. Document Parsing

Raw documents cannot be directly used by the LLM. So we first parse them and extract clean text.

Example:

From a PDF, we extract: Title, headings, paragraphs, tables, sections, and important metadata

Poor parsing can break the entire RAG quality because the model may receive incomplete or messy context.

4. Chunking

Chunking means breaking large documents into smaller meaningful pieces.

Example: A 50-page policy document can be split into smaller chunks like:

- Refund Policy section
- Cancellation Policy section
- Premium Customer Policy section
- Payment Terms section

Why chunking is needed?

Because LLMs cannot always process very large documents at once, and vector search works better when the content is divided into meaningful smaller blocks.

Core Components of a RAG Pipeline

5. Embedding Model

An embedding model converts text into numerical vectors.

Example:

Text: “Premium customers are eligible for refund within 30 days.” gets converted into a vector like: [0.12, -0.45, 0.88, 0.31, ...] , These vectors help us compare meaning, not just keywords.

So even if the user asks: “Can premium users get their money back?” the system can still match it with the refund policy document.

6. Vector Database / Vector Store

A vector database stores embeddings of all document chunks.

Examples:

Amazon S3 Vector Bucket, OpenSearch, Pinecone, Weaviate, ChromaDB, FAISS, pgvector

When the user asks a question, the vector database helps find the most semantically similar chunks.

Think of it as a search engine for meaning.

Core Components of a RAG Pipeline

7. User Query

This is the question asked by the user.

Example:

“What is the refund policy for premium customers?”

The query is also converted into embeddings so it can be compared with stored document embeddings.

8. Retriever

The retriever searches the vector database and brings back the most relevant chunks.

Example retrieved chunks:

- Chunk 1: Premium customers can request a full refund within 30 days.
- Chunk 2: Refunds are not applicable after 30 days.
- Chunk 3: Refund requests must be raised from the billing dashboard.

The retriever decides what context should be sent to the LLM.

Core Components of a RAG Pipeline

9. Prompt Builder

The prompt builder combines: User question + Retrieved context + Instructions

Example:

User Question: What is the refund policy for premium customers?

Retrieved Context: Premium customers can request a full refund within 30 days...

Instruction to LLM: Answer only using the provided context. Do not make assumptions.

This step helps control the LLM response and reduces hallucination.

10. LLM / Foundation Model

The LLM generates the final answer using the retrieved context.

Examples: Claude Opus, Amazon Nova, Llama, Mistral, GPT models

The LLM is not randomly answering from memory. It is answering based on the retrieved business context.

Core Components of a RAG Pipeline

9. Prompt Builder

The prompt builder combines: User question + Retrieved context + Instructions

Example:

User Question: What is the refund policy for premium customers?

Retrieved Context: Premium customers can request a full refund within 30 days...

Instruction to LLM: Answer only using the provided context. Do not make assumptions.

This step helps control the LLM response and reduces hallucination.

10. LLM / Foundation Model

The LLM generates the final answer using the retrieved context.

Examples: Claude Opus, Amazon Nova, Llama, Mistral, GPT models

The LLM is not randomly answering from memory. It is answering based on the retrieved business context.